

AKSHAR AI

SUMMER INTERNSHIP REPORT

WEEK 7 | 04 JULY 2026 – 10 JULY 2026

Project Title: Pneumonia Detection System & Human Behavior Tracking System

DETAILS

Name: Yashas Raj R

Enrollment No.: 24162121033

Branch: BDA (B.Tech – Computer Science and Engineering)

Institute: ICT Ganpat University, Mehsana, Gujarat

Organisation: Akshar AI Ltd

Report Period: 04 July 2026 – 10 July 2026 (Week 7)

Email: 24162121033@gnu.ac.in

Project Title: Pneumonia Detection System / Hand & Gesture Recognition

1. Introduction

During Week 7, the primary development phase for both the Pneumonia Detection System and the Hand Tracking System was completed. The underlying architectures, models, and web integrations were in place. Therefore, the focus for this week shifted to quality assurance, testing, bug fixing, and performance optimization.

Transitioning from a working prototype to a stable application requires thorough testing. Models that perform well in a controlled development environment often face challenges when deployed in a web application. Week 7 was dedicated to identifying and resolving these integration issues.

For the Pneumonia Detection System, testing involved checking how the Flask backend API handled edge cases, such as invalid file formats and large image sizes. At the same time, the frontend interface was refined to improve usability and responsiveness across different devices.

For the Hand & Gesture Recognition Module, testing focused on the reliability of the gesture classification algorithms. The MediaPipe integration was tested under various lighting and movement conditions to identify any limitations and ensure the gesture classification (Open Palm vs. Fist) remained accurate and stable.

This report outlines the testing methods used, the bugs identified, and the solutions implemented during Week 7, highlighting the validation processes necessary to prepare the software for practical use.

2. Weekly Objectives

The objectives for Week 7 focused on stabilizing the system, fixing bugs, and improving the user experience.

The primary objectives were as follows:

1. **Backend Testing (Flask API):** Test the /predict endpoint with various edge cases to evaluate how the server handles incorrect file types and large uploads.
2. **Frontend Error Handling & UX:** Update the frontend JavaScript to properly handle backend errors, ensuring users receive clear and helpful error messages.
3. **UI/UX Refinement:** Adjust the CSS styling of the web application for better responsiveness on mobile devices and improved accessibility.
4. **Gesture Tracking Testing:** Test the hand tracking module under different environmental conditions, such as low light and fast movement, to evaluate accuracy.
5. **Output Stabilization:** Implement debouncing logic in the gesture classification to prevent the output from flickering when the hand is near the decision threshold.

6. **Codebase Cleanup:** Review and clean the repository by removing obsolete test files, organizing CSS, and ensuring Python scripts follow standard formatting guidelines.

3. Technical Implementation Details (Backend)

The initial version of the Flask backend assumed users would upload correctly formatted images. During Week 7, the code was updated to handle unexpected inputs more securely.

Input Validation: Strict MIME type checking was added before the image is processed by the Pillow library. If an unsupported file type is uploaded, the server now rejects the request and returns a 415 Unsupported Media Type error. Additionally, error handling was added around the image opening process to catch corrupted files gracefully, preventing the server from crashing and returning a standard error message instead.

Memory Management: To prevent the server from running out of memory when processing exceptionally large image files, a strict size limit was enforced on the server-side. Uploads exceeding 5MB are intercepted early in the request lifecycle, ensuring server resources remain available for valid requests.

4. Technical Implementation Details (Frontend)

The frontend JavaScript was updated to better communicate backend issues to the user.

Graceful Error Display: Previously, server errors could cause the frontend loading indicator to hang. The fetch() logic was updated to check for failed network requests or non-200 status codes. If an error occurs, the UI now stops the loading indicator and displays an alert box with the specific error message provided by the server. This ensures the user is informed of the issue.

Responsive Design Adjustments: CSS Media Queries were thoroughly reviewed and updated. The interface now dynamically stacks elements vertically on smaller screens (mobile view) while maintaining a side-by-side layout on desktop monitors. Button padding and font sizes were adjusted for better touch interaction on mobile devices.

5. Technical Implementation Details (Gesture Recognition)

During testing of the Hand Tracking module, the classification output occasionally flickered between “Open Palm” and “Closed Fist” when the user's hand was near the threshold point, primarily due to minor camera sensor noise.

Implementing Temporal Smoothing: To fix this, a debouncing algorithm was added. The system now maintains a brief buffer of the most recent frames (e.g., the last 10 frames). Using a deque data structure, the oldest frame is discarded as a new one arrives. The final classification displayed on screen is based on the statistical mode (the most common result) within that buffer, smoothing out momentary inconsistencies and providing a stable output.

6. Day-wise Work Summary: Day 1 (Monday)

Focus: Backend Edge-Case Testing

The week started with testing the Flask backend using various invalid files, such as documents renamed as images and excessively large image files. I added file extension validation and MIME type checking to properly reject these files. Error handling was also improved to prevent the server from crashing when processing difficult files. The API now returns appropriate HTTP error codes for invalid inputs.

I wrote comprehensive test cases to ensure that:

- Files with spoofed extensions (e.g., a PDF named image.jpg) are rejected.
- Files exceeding 5MB return an immediate error response.
- Missing files in the request payload return a standard bad request response.

7. Day-wise Work Summary: Day 2 (Tuesday)

Focus: Frontend Error Management

Tuesday focused on updating the frontend to handle the new backend error codes. I updated the JavaScript `fetch()` function to check the response status. If the backend returns an error (like a 415 or 422), the frontend now stops the loading animation and displays a clear error message on the screen, improving the overall user experience.

I also ensured that network timeouts and CORS (Cross-Origin Resource Sharing) errors were caught in the main catch block of the Promise chain, ensuring the interface never enters an infinite loading state.

8. Day-wise Work Summary: Day 3 (Wednesday)

Focus: Environmental Testing (Computer Vision)

On Wednesday, I tested the Hand Tracking application under different physical conditions. I tested it in strong backlight, low light, and with rapid hand movements. I found that strong backlight

affected the initial palm detection, and fast movement slightly reduced the accuracy of the 3D landmarks.

These findings were documented to outline the optimal operating conditions for the module. I recorded the minimum lux (lighting) levels required for the MediaPipe engine to accurately render the 21 3D spatial points.

9. Day-wise Work Summary: Day 4 (Thursday)

Focus: Implementing Temporal Smoothing

To resolve the flickering issue observed during gesture classification, I implemented a smoothing buffer on Thursday. Using a `collections.deque` in Python, the system tracks a short history of frames and outputs the most common classification.

This small addition made the real-time tracking significantly more stable and consistent. I experimented with different buffer sizes (5, 10, and 15 frames) and determined that a 10-frame buffer provided the best balance between stability and perceived real-time responsiveness.

10. Day-wise Work Summary: Day 5 (Friday)

Focus: UI/UX Refinement

Friday was spent improving the visual design of the web application. I reviewed the CSS file and added media queries to ensure the layout adapted correctly to mobile and tablet screens. Minor adjustments were also made to colors and button styles to make the interface cleaner and more accessible.

I utilized standard accessibility guidelines to verify that text contrast was sufficient and that form elements were properly labeled for screen readers.

11. Day-wise Work Summary: Day 6 (Saturday)

Focus: Codebase Cleanup

The final day of the week was dedicated to organizing the project repository. I removed old experimental scripts and unnecessary files that had accumulated during development.

I also ran a linter (Flake8) on the Python code to ensure formatting was consistent with standard guidelines. This helps keep the codebase readable and maintainable for future developers. All CSS rules were alphabetized, and redundant classes were consolidated.

12. Quality Assurance Methodology

The quality assurance phase followed standard software testing principles:

- **Unit Testing:** Ensuring individual utility functions (like the Euclidean distance calculator) performed correctly under varying mathematical inputs.
- **Integration Testing:** Ensuring the Flask backend and the HTML frontend communicated correctly over HTTP, particularly when handling Multipart Form Data.
- **Stress Testing:** Pushing the system beyond its intended operational parameters to observe failure modes and ensure safe degradation.

13. Challenges and Resolutions

- **Challenge:** Handling asynchronous state changes in the UI.
- **Resolution:** Careful management of CSS classes during the JavaScript Promise lifecycle to clearly indicate Loading, Error, and Success states.
- **Challenge:** Sensor noise causing classification flickering.
- **Resolution:** Implementation of the statistical mode algorithm over a rolling frame buffer.

14. Conclusion

Week 7 was an important phase for consolidating and refining the project. The testing and bug-fixing efforts during this week helped ensure that the applications were stable and reliable.

The Pneumonia Detection System's backend was improved to handle edge cases properly, and the frontend was updated to provide better feedback to the user. The Hand Tracking module was stabilized, resulting in consistent and accurate real-time classifications. With the main issues resolved and the codebase organized, the project is well-prepared for final documentation and presentation in the upcoming week.

15. Appendix and References

- **Flask Documentation:** Used for implementing secure file upload handling.
- **MediaPipe Solutions:** Referenced for optimal camera configurations and landmark tracking parameters.

- **MDN Web Docs:** Utilized for modern JavaScript Fetch API patterns and Promise management.
- **PEP 8 Guidelines:** Adhered to for final Python code formatting.